

AcousticX Intelligence · Sound-Event Detection System

SonicSentinel AI

Technical Requirements, MoSCoW Prioritization & Delivery Roadmap



Prepared for: Aptech TechWiz 7 — NextWave AI and ML Category
Document Type: Requirements Analysis, MoSCoW Prioritization & Delivery Roadmap
Version 1.0

1. Introduction — What This Document Is

This document takes the official Software Requirements Specification (SRS) for **SonicSentinel AI** and turns it into a practical project-management plan. Its job is to answer three questions for anyone on the team, from a first-year developer to a non-technical judge: **what exactly are we building, why does each piece matter, and in what order should we build it?**

In plain words

SonicSentinel AI is a website + AI system that "listens" to sound — either a recording you upload or your live microphone — and tells you what it heard: a gunshot, glass breaking, a scream, machinery trouble, or just normal background noise. If it hears something dangerous, it raises an alert, the same way a smoke detector raises an alarm when it senses smoke, except this one "smells" sound instead of smoke, and it double-checks itself using two independent AI models before it decides to sound the alarm.

To turn the 80 functional requirements and 5 non-functional requirements listed in the SRS into a buildable plan, this document uses two well-known project-management techniques:

- **MoSCoW Prioritization** — every requirement is sorted into **Must Have**, **Should Have**, **Could Have**, or **Won't Have (this phase)**, so the team always knows what to build first.
- **Phased / MVP Roadmap** (a lightweight form of Agile release planning) — the Must Have items become "Release 1 (Minimum Viable Product)", Should Have items become "Release 2", and Could Have items become "Release 3 / stretch goals".

2. The Problem, Explained Like You're Not a Developer

2.1 Why does this app need to exist?

Factories, warehouses, offices, and public spaces are full of sounds all day. Right now, if something dangerous happens — a window breaks, a machine starts failing, someone screams for help — the only way anyone finds out is if a human happens to be listening at that exact moment, or if someone reviews hours of recorded audio afterwards. That is slow, tiring, and things get missed, especially in noisy or busy places.

2.2 What SonicSentinel AI does about it

SonicSentinel AI replaces the "human constantly listening" job with software. It listens to audio (uploaded files or a live microphone) 24/7, automatically figures out what kind of sound it is, and immediately raises an alert if it is something serious. It never gets tired, never gets distracted, and keeps a permanent written record of everything it hears.

2.3 Who actually uses it (the people, in plain terms)

Role	Who this represents	What they mainly do in the app
Normal User	Any employee or resident	Uploads clips, starts the mic, sees basic detections that concern them
Audio Reviewer	A staff member who checks uncertain AI results	Listens to flagged clips and confirms or corrects the AI's guess
Security Operator	Guard / security desk staff	Watches the live dashboard, responds to Critical/High alerts
Maintenance Operator	Facilities / engineering staff	Responds to Machinery Fault alerts and inspection recommendations
Administrator	System owner / IT manager	Manages users, configures alert rules and thresholds, sees system-wide analytics

3. How The System Works — Step by Step, In Plain English

Think of this as an assembly line for sound, with two independent "inspectors" checking each piece of sound before a final decision is made.

1. **Sound comes in.** Either the user uploads a file, or the live microphone is switched on.
2. **Quality check.** The app makes sure the audio is real, not broken, not silent, and in a supported format — like a bouncer checking ID before letting someone into a club.
3. **Clean-up (pre-processing).** The app removes some background hiss, adjusts the volume to a standard level, and slices the audio into short, equal-length pieces.
4. **Feature extraction.** The app converts each short sound clip into a set of numbers that describe its "shape" — pitch, texture, energy pattern — because AI models understand numbers, not raw sound waves.
5. **Two independent AI opinions.** The *same* cleaned-up sound is shown to two completely separate AI models: one built and trained in Python by the team, and one trained using Google's Teachable Machine tool. Neither model is told what the other one thinks.
6. **Compare the two opinions.** The app checks: did both models agree on the category? How close were their confidence percentages? If they agree confidently, that's a strong result. If they disagree, or both are unsure, the result is marked uncertain.
7. **Apply safety rules.** For dangerous sounds (gunshot, scream, glass breaking, etc.), the app also checks: was this heard more than once? Was audio quality good enough to trust? Only if all these checks pass does the app treat the event as confirmed.
8. **Decide and act.** The system assigns a severity level, and if it's serious, fires a real-time alert on the dashboard. If anything about the result was shaky, it goes into a queue for a human reviewer instead of being trusted blindly.
9. **Record everything.** Every step, every score, and every decision is saved so it can be searched, reported on, and reviewed later.

Why two AI models instead of one?

Just like a hospital might want a second doctor's opinion before a serious diagnosis, using two independently-trained models means the system is much less likely to be fooled by one model's blind spot. If both agree, we can be far more confident. If they disagree, that disagreement itself is useful information — it tells the system "don't trust this one, get a human."

4. Functional Requirements — MoSCoW Prioritized

The SRS lists 80 functional requirements. Below, every single one is restated in plain language and sorted using the **MoSCoW method**:

MoSCoW, explained simply

Must Have: the product is broken or pointless without this. **Should Have:** important and expected, but the product can go live without it and add it soon after. **Could Have:** a genuinely nice extra if there's spare time. **Won't Have (this phase):** intentionally left out for now, to be revisited in a future version.

MUST HAVE — Core, non-negotiable for a working product — 42 requirements

#	Requirement	What this means in plain English	Priority	Why this priority
i	User Registration & Authentication	Anyone using the app (a normal user, a security guard, a maintenance person, or an admin) needs to create an account and log in safely, like logging into an email account.	MUST HAVE	Nobody can use any other feature of the app without an account. This is the front door.
ii	Role-Based Access Control	Different people see different things. A maintenance worker only sees machine-fault alerts; a security guard sees glass-breaking/gunshot alerts; an admin sees everything and can change settings.	MUST HAVE	Without this, anyone could see or change everything, which is unsafe and confusing.
iv	Audio File Upload	A user can pick an audio file (WAV, MP3, FLAC, OGG, or M4A) from their computer and send it to the app for checking.	MUST HAVE	This is one of the two main ways sound gets into the system - the whole product is built around it.
vi	Live Microphone Monitoring	The app can 'listen' through a connected microphone in real time and analyse sound as it happens, like a live security camera but for sound.	MUST HAVE	This is the second core input method and is central to the product's promise of real-time protection.
vii	Microphone Status Display	The screen clearly shows if the mic is on, paused, disconnected, or if permission was refused - so the user always knows if they are actually being monitored.	MUST HAVE	If users can't tell whether the mic is really listening, they may falsely believe they are protected when they are not.
viii	Audio File Validation	Before analysing a file, the app checks it is a real, usable audio file (right format, size, length, not broken, not silent).	MUST HAVE	Feeding a broken or empty file into the AI models would produce garbage results and could crash the app.
x	Audio Metadata Extraction	The app reads and displays basic facts about the file - its name, format, length, quality settings, size, and when it was uploaded.	MUST HAVE	This information is needed later for reports, audits, and to help the AI models process the file correctly.
xi	Audio Pre-processing	Before the AI looks at the sound, the app cleans it up: converts stereo to mono, adjusts volume levels, trims silence, reduces background hiss, and cuts it into equal-length chunks.	MUST HAVE	AI models are trained on cleanly-formatted audio; skipping this step badly hurts detection accuracy.
xii	Silence Detection	The app notices when a clip is basically silent and stops it from being wrongly labelled as a real sound event.	MUST HAVE	Otherwise the system could raise false alerts from empty recordings, which destroys user trust.
xv	Audio Segmentation	Long recordings or a continuous microphone stream are automatically cut into short, fixed-length pieces (e.g. 1-3 seconds) that are easier for the AI to analyse one at a time.	MUST HAVE	Real-time detection and long-file analysis are both impossible without cutting audio into manageable chunks.
xvi	Common Sound Dataset	One shared library of labelled example sounds (machinery fault, glass breaking, gunshot, etc.) is used to teach both AI models.	MUST HAVE	Both AI models must learn from the same examples, otherwise their answers cannot be fairly compared.
xvii	Dataset Metadata Management	Every sound sample in the training library keeps a record card: an ID, filename, category, source, length, recording conditions, and whether it's for training/validation/testing.	MUST HAVE	Needed to prove the dataset is organised correctly and to avoid mixing training and testing samples by mistake.
xx	Acoustic Feature Extraction	Instead of feeding raw sound waves into the model, the app calculates measurable 'fingerprints' of the sound (like pitch pattern, energy, and tone texture) that the model actually learns from.	MUST HAVE	Nearly every practical audio-classification model depends on these calculated features - without them, classification does not work.
xxiii	Python Model Training	The team must build, train, and compare at least three different machine-learning approaches to find the one that best recognises the ten sound categories.	MUST HAVE	This is literally the 'brain' of the app - without it there is no AI-based detection at all.
xxv	Python Sound Classification	The trained Python model looks at a segment of audio and decides which one of the ten sound categories it most likely is.	MUST HAVE	This is the core output the entire application exists to produce.

xxvi	Python Confidence Scores	Along with its main answer, the Python model also reports how sure it is about every possible category (e.g. 92% Glass Breaking, 5% Alarm, 3% other).	MUST HAVE	Confidence numbers are needed for alert rules, manual review triggers, and comparing against the second model.
xxvii	Google Teachable Machine (GTM) Audio Model Training	A second, separately-trained AI model is built using Google's free Teachable Machine tool, learning from the same sound library.	MUST HAVE	The SRS requires two independent 'opinions' to cross-check each other, which is central to the whole design.
xxviii	GTM Model Integration	The trained Teachable Machine model is connected into the website so it can be used automatically, not just tested manually on Google's site.	MUST HAVE	Without integration, the second model exists only as a demo and cannot contribute to real decisions.
xxix	GTM Sound Classification	The Teachable Machine model independently looks at the exact same audio segment and makes its own guess, without seeing the Python model's answer.	MUST HAVE	Independence between the two models is what makes the cross-checking meaningful and trustworthy.
xxx	GTM Confidence Scores	The Teachable Machine model also reports how confident it is in each of the ten categories.	MUST HAVE	Needed to calculate agreement/disagreement between the two models.
xxxi	Prediction Comparison	The app checks whether the Python model and the Teachable Machine model agree on the sound category.	MUST HAVE	Agreement between two independent models is the app's main way of trusting a result before alerting anyone.
xxxii	Confidence Comparison	The app calculates the numeric gap between the two models' confidence levels for their top answer.	MUST HAVE	A big gap signals the result is shakier and may need a human to double-check it.
xxxiii	Model Consistency Status	The app labels every result as 'Acceptable Match', 'Weak Match', 'Model Disagreement', or 'Uncertain Result' so a human can understand at a glance how trustworthy it is.	MUST HAVE	Turns the raw comparison numbers into something a non-technical user can instantly understand.
xxxvii	Audio Quality Classification	Every clip gets labelled Good, Acceptable, Poor, or Unusable based on how clean the recording is.	MUST HAVE	Alert rules, manual review routing, and trust in results all depend on knowing the recording quality.
xxxviii	Uncertain Sound Detection	The app flags a result as 'uncertain' whenever confidence is low, the two models disagree, quality is poor, or sounds overlap.	MUST HAVE	This is the main safety net that prevents the system from confidently reporting a wrong answer.
xl	Repeated Event Confirmation	Before declaring a dangerous event (like a gunshot) real, the app checks it was detected more than once, or across consecutive time windows, rather than trusting a single blip.	MUST HAVE	A single false spike triggering a 'gunshot' alert would seriously damage user trust in a security product.
xli	Machinery Fault Detection	When the machine sound looks abnormal, the app raises a maintenance alert suggesting an inspection.	MUST HAVE	This is one of the ten required sound categories the whole product must reliably detect.
xlii	Glass Breaking Detection	When glass-breaking sound is detected with enough confidence, a high-priority security alert is raised.	MUST HAVE	One of the ten mandatory categories and a key security use-case for the product.
xliii	Alarm or Siren Detection	Detected alarms/sirens raise a high-priority 'pay attention' alert.	MUST HAVE	Mandatory category; alarms often indicate an emergency already underway elsewhere.
xlvi	Gunshot Detection	Only after passing extra confirmation rules (repeat detection, high confidence, model agreement) does the app declare a 'Critical' gunshot alert.	MUST HAVE	This is the single most safety-critical detection the product performs and must be handled carefully.
xlvii	Panic-Scream Detection	A scream detected with sufficient confidence raises a Critical safety alert.	MUST HAVE	Mandatory category directly tied to protecting people in danger - a core promise of the product.
xlviii	Aggression Detection	Shouting/fighting sounds raise a High or Critical alert depending on how confident and how repeated the detection is.	MUST HAVE	Mandatory category central to the security use-case of the app.
xlix	Person Asking for Help Detection	The app listens for specific trained help-phrases (like 'Help me', 'Emergency') and raises a Critical alert when heard.	MUST HAVE	This is arguably the most human-impact category - directly detecting someone in distress.
lii	Alert Severity Assignment	Every alert is tagged with a severity level: Informational, Low, Medium, High, or Critical, so responders know how urgently to react.	MUST HAVE	Without severity levels, every alert looks equally urgent, which overwhelms the people responding.
liv	Real-Time Alert Generation	The moment a critical event is confirmed, a clearly visible alert pops up on the live dashboard.	MUST HAVE	Real-time alerting is the entire point of a live monitoring security product.
lvii	Manual Review Queue	Any result the AI isn't confident about is automatically placed into a queue where a human double-checks it.	MUST HAVE	This is the safety valve of the whole design - it stops uncertain AI guesses from being treated as fact.

Ixiii	User Dashboard	A normal user's homepage shows their recent uploads, current detections, critical events, quality warnings, and anything awaiting review.	MUST HAVE	Users need a home screen to actually see and act on the results the AI produces.
Ixiv	Live Monitoring Dashboard	A dedicated live screen shows the microphone's status, the current detected sound, both models' answers, agreement status, severity, and any active alert - updating in real time.	MUST HAVE	This is the centrepiece screen for the app's live-monitoring promise.
Ixxi	Secure Audio Storage	Uploaded and recorded audio clips are stored safely with controlled access so only authorised people can retrieve them.	MUST HAVE	Handling potentially sensitive recordings (e.g. of a panic scream) responsibly is a basic trust and safety requirement.
Ixxii	Database Storage	All the important data - users, audio details, predictions, confidence scores, alerts, reviews, model versions, and audit logs - is stored in an organised database.	MUST HAVE	Nothing in the app (dashboards, history, reports) works without a proper place to store this information.
Ixxvii	Error Handling	When something goes wrong (a bad file, an unsupported format, a broken model, a database hiccup), the app shows a clear, understandable message instead of crashing or showing a scary technical error.	MUST HAVE	A product that crashes on the first bad input will fail live demonstrations and frustrate real users.
Ixxix	Privacy Controls	Users are always shown clearly when the microphone is actively listening, and the app is never allowed to record secretly.	MUST HAVE	Secretly recording people is both unethical and against the SRS's own explicit rules - this is non-negotiable.

SHOULD HAVE — Important, adds real value, but the app can launch without it on day one — 32 requirements

#	Requirement	What this means in plain English	Priority	Why this priority
iii	User Profile Management	Users can edit their own name, contact details, etc., and the system gives every user a unique ID number.	SHOULD HAVE	Useful for record-keeping but the app can technically detect sounds without a polished profile page.
ix	Audio Preview	A user can press play/pause, skip around, and adjust volume to listen to an uploaded clip before or after analysis.	SHOULD HAVE	Nice for reviewing evidence, but analysis can happen without a fancy audio player.
xiii	Clipping Detection	The app spots recordings that were too loud for the microphone and got distorted, and warns the user about it.	SHOULD HAVE	Improves reliability and gives context to a strange result, but the app can still attempt a prediction without it.
xiv	Background Noise Estimation	The app measures how much constant background noise (traffic, fans, chatter) is in a clip.	SHOULD HAVE	Useful context for judging how much to trust a result, but not required for a first working version.
xviii	Dataset Balance Checking	The system checks that there are roughly equal numbers of examples for each sound category, and flags it if one category has too few.	SHOULD HAVE	Improves model fairness/accuracy but the project can start with a manually-balanced dataset.
xix	Audio Augmentation	The training pipeline can automatically create realistic variations of a sound (adding noise, shifting pitch, changing volume) to expand the dataset without recording new audio.	SHOULD HAVE	Boosts accuracy and robustness, but the models can be trained on real recordings alone first.
xxi	Waveform Generation	The app draws a picture of the sound's volume over time (the up-and-down wavy line you see in audio editors).	SHOULD HAVE	Helpful for humans reviewing a clip, but the AI's decision does not depend on the picture being shown.
xxii	Spectrogram Generation	The app draws a colour-coded picture showing which pitches/frequencies were present in the sound over time.	SHOULD HAVE	Same as waveform - a helpful visual for human reviewers, not required for the AI decision itself.
xxiv	Hyperparameter Tuning	The team systematically tests different internal settings of the chosen model (like a recipe's oven temperature and cooking time) to get the best accuracy.	SHOULD HAVE	Improves accuracy incrementally; a first working model can run on reasonable default settings.
xxxiv	Top-N Prediction Display	Instead of showing only the winning guess, the screen also shows the next two most likely sound categories from each model.	SHOULD HAVE	Gives extra insight for reviewers but is not essential for basic alerting.
xxxv	Configurable Confidence Threshold	An administrator can set the minimum confidence percentage needed before the system automatically raises an alert.	SHOULD HAVE	Improves tuning flexibility; the system can launch with a sensible fixed default.
xxxvi	Top-Two Margin Threshold	Administrators can set how much higher the top guess must be than the second-best guess before the app trusts the result.	SHOULD HAVE	Fine-tuning control; a reasonable default value works for an initial release.
xxxix	Overlapping Sound Detection	The app notices when several different sound categories all got meaningful confidence at once (e.g. a scream over machine noise), meaning more than one thing may be happening.	SHOULD HAVE	Adds nuance to complex real-world audio; a first version can treat this as a general 'uncertain' case.

xliv	Vehicle-Horn Detection	Vehicle horns are logged as ordinary traffic/environmental sounds rather than security threats.	SHOULD HAVE	Needed for completeness of the ten categories, but it is a low-risk, informational-only event.
xlv	Animal-Sound Detection	Animal sounds (barking, etc.) are logged but generally treated as low concern unless context says otherwise.	SHOULD HAVE	Low-risk category; useful to log but not urgent to perfect early.
I	Background Noise Handling	Ordinary background hum is treated as harmless unless it becomes unusually loud/long, in which case it's flagged.	SHOULD HAVE	Important for reducing noise/false alerts, but a simple default rule is enough to start.
li	Unknown Sound Handling	Anything the AI can't confidently classify is labelled 'Unknown' or sent for a human to review rather than guessed at.	SHOULD HAVE	Prevents wrong guesses on unfamiliar sounds; can initially fall back to the general manual-review queue.
liii	Configurable Alert Rules	Administrators can adjust which categories count as critical, what confidence is needed, and what action is recommended.	SHOULD HAVE	Powerful for customisation later, but the app can launch with sensible built-in rules.
lv	Alert Acknowledgement	Authorized staff can mark an alert as seen, dismiss it, or escalate it further.	SHOULD HAVE	Improves workflow management but the raw alert is already useful without this extra step.
lvi	Alert History	A full, searchable log of every past alert and what staff did about it is kept.	SHOULD HAVE	Valuable for audits and learning from past events, but not required for the very first release.
lviii	Reviewer Audio Playback	A human reviewer can listen to the exact audio clip in question while deciding what it really was.	SHOULD HAVE	Makes manual review much easier and more accurate, though a text summary alone could work at a pinch.
lix	Reviewer Decision	The reviewer can confirm the AI's guess was right, or correct it to the true category.	SHOULD HAVE	Improves data quality over time, but isn't required for a first functioning review queue.
lxi	Decision Override	A reviewer can overrule what the AI decided while the original AI answer is still kept on record for reference.	SHOULD HAVE	Important for trust and accountability, but the system can work with review-and-flag only at first.
lxii	Event Status Tracking	Every detected sound event moves through clear stages - Uploaded, Classified, Uncertain, Alert Generated, Manual Review, Reviewed, Closed.	SHOULD HAVE	Helps organise workflow but a simpler 'processed / needs review' state is enough for a first cut.
lxv	Administrator Dashboard	Admins see system-wide statistics: total events, breakdown by category, critical alerts, average confidence, disagreements, poor-quality recordings, and trends over time.	SHOULD HAVE	Very useful for oversight, but the app can function operationally before this is fully built.
lxvi	Event Timeline	Detected events are shown in date/time order so patterns over a day or week can be seen.	SHOULD HAVE	Nice-to-have visual, achievable with a simple sorted list initially.
lxvii	Search and Filtering	Users can filter recorded events by ID, filename, category, date, confidence range, severity, quality, review status, or user.	SHOULD HAVE	Very useful once there are many records, but not needed to prove the core detection works.
lxix	Downloadable Analysis Report	A single event's full details (metadata, both predictions, confidence, waveform, spectrogram, quality, severity, decision) can be exported as a document.	SHOULD HAVE	Important for the competition's evidence requirements, achievable once the core data exists.
lxxiii	Duplicate Audio Detection	The app calculates a digital fingerprint of every uploaded file to notice when the exact same recording is uploaded twice.	SHOULD HAVE	Keeps the record-keeping clean, useful but not essential for the AI detection itself to work.
lxxv	Model Version Tracking	Every prediction stores exactly which version of the Python model and the Teachable Machine model produced it.	SHOULD HAVE	Important for fair audits and explaining old results after a model is later retrained.
lxxvi	Audit Trail	The app keeps a log of logins, uploads, mic sessions, predictions, alerts, reviews, overrides, exports, and model updates.	SHOULD HAVE	Needed for accountability and to satisfy the competition's integrity checks, but the core detection can be demoed without a full audit log first.
lxxx	Data Retention	Administrators can configure how long audio files and event records are kept before being deleted.	SHOULD HAVE	Good governance practice, but a first version can keep everything and add auto-deletion later.

COULD HAVE — Nice extras if time and resources allow — 6 requirements

#	Requirement	What this means in plain English	Priority	Why this priority
v	Batch Audio Upload	Instead of uploading one file at a time, a user can upload many files together and get results for all of them.	COULD HAVE	Convenient for power users, but the core detection works fine one file at a time.

ix	Reviewer Comments	Reviewers can type free-text notes and suggested next steps on a reviewed event.	COULD HAVE	A helpful extra for documentation, not essential to basic review functioning.
lxviii	Analytics and Reporting	The app calculates statistics such as how often each category appears, false-positive/negative rates, and audio quality trends.	COULD HAVE	Valuable for long-term evaluation of the system, but this is analysis on top of a working core, not core itself.
lxx	Data Export	Administrators can export all records as a CSV or Excel file for further analysis outside the app.	COULD HAVE	Useful add-on for advanced users, not required for the app to function day-to-day.
lxxiv	Near-Duplicate Detection	The app also tries to catch recordings that are almost the same file, just re-saved, trimmed, or with volume changed.	COULD HAVE	A more advanced, 'nice extra' version of duplicate checking.
lxxviii	Monitoring and Anomaly Alerts	Administrators are separately warned about system-level problems like repeated failed uploads, model failures, unusual result spikes, or many failed logins.	COULD HAVE	A mature 'ops' feature for running the system long-term; not required to prove the core idea works.

5. Non-Functional Requirements — What "Good" Looks Like

Functional requirements describe *what* the app does. Non-functional requirements describe *how well* it needs to do it — things a user would notice even without being told, like speed and reliability.

#	Requirement	What this means in plain English	Priority	Why this priority
1	Performance	Any uploaded clip up to 30 seconds must get a result within 8 seconds; a live microphone window must get a result within 3 seconds.	MUST HAVE	If the app is slow, it fails at being a 'real-time' safety tool - speed is part of the core promise.
2	Scalability	The system must be able to store at least 20,000 sound events and serve several users at the same time without slowing down.	SHOULD HAVE	Needed to be usable in a real deployment, but a smaller-scale demo can prove the concept first.
3	Usability	The interface must be clear and easy to use for every type of user - ordinary users, reviewers, security staff, maintenance staff, and admins.	MUST HAVE	A confusing interface undermines even a technically perfect AI - people need to actually understand and trust it.
4	Accuracy	Both AI models should reach at least 85% overall accuracy and 0.80 macro F1-score; the five most dangerous categories (gunshot, glass breaking, panic scream, aggression, calls for help) should catch at least 85% of true cases (recall).	MUST HAVE	The whole point of the app is correct detection - missing real emergencies defeats its purpose.
5	Availability	The app should stay online and working at least 99% of the time during monitoring/evaluation hours.	SHOULD HAVE	Important for a production security tool, though a working prototype can be demonstrated with slightly less formal uptime guarantees.

6. Delivery Roadmap — A Second Prioritization Method

MoSCoW tells us *what matters most*. To turn that into an actual build schedule, we also apply a simple **phased / MVP (Minimum Viable Product) roadmap**, a widely-used Agile planning technique. Each phase is a working, demoable version of the app — it just gets more complete each time.

Phase 1 — Minimum Viable Product (build all "Must Have" items)

Goal: a working end-to-end pipeline that can take audio in, classify it with both AI models, compare them, apply critical-event safety rules, and show a live alert. This alone proves the whole concept works.

- Accounts, login, and role-based access
- Upload + live microphone capture, with validation and clean-up
- Feature extraction, and both AI models trained and connected
- Prediction & confidence comparison, consistency status
- Repeated-detection confirmation and severity-based alerts for all ten sound categories
- Manual review queue for uncertain results
- Basic User Dashboard and Live Monitoring Dashboard
- Secure storage, database, and clear error handling
- Microphone privacy indicator (never secretly recording)

Phase 2 — Should Have (polish and completeness)

- Profile management, audio preview, quality/clipping/noise indicators
- Dataset balancing and augmentation tools; waveform/spectrogram visuals
- Hyperparameter tuning; configurable thresholds and alert rules
- Alert acknowledgement, alert history, event status tracking
- Reviewer playback, decisions, and override capability
- Administrator Dashboard, event timeline, search & filtering
- Downloadable reports, duplicate-audio detection, model version tracking, audit trail
- Configurable data retention

Phase 3 — Could Have (stretch goals, if time allows)

- Batch upload, reviewer comments
- Full analytics/reporting suite, CSV/Excel data export
- Near-duplicate audio detection
- System-health monitoring and anomaly alerts for admins

7. Dataset Requirements, Explained Simply

An AI model only learns from examples, so the team must build a library of real (or ethically collected) sound clips before any training can start.

- **Size:** at least 3,000 unique original clips in total.
- **Balance:** about 300 clips for each of the ten required categories (machinery fault, glass breaking, alarm/siren, vehicle horn, animal sound, gunshot, panic scream, aggression, person asking for help, background noise).
- **Variety:** different devices, distances, indoor/outdoor settings, loudness, and clean vs. noisy conditions — so the AI doesn't just memorise one specific microphone's sound.
- **Splitting the data:** 70% is used to actually teach the model (Training), 15% is used to check progress while building it (Validation), and 15% is kept completely hidden until the very end to test real performance (Testing) — this prevents the model from "cheating" by memorising the test questions in advance.
- **"Person Asking for Help" clips** must use a defined, limited set of safety phrases like "Help me" or "Emergency", collected ethically (recorded voluntarily or sourced with permission).

8. Competition Integrity Rules, Explained Simply

These rules exist to make sure the finished project reflects genuine understanding, not shortcuts or copy-pasted answers.

- Every team member must be able to explain the parts of the app they personally built.
- Work must be committed to GitHub regularly across all days, not dumped in at the last minute.
- A daily development log must record what was done, what broke, and how it was fixed.
- On the final day, the team may be asked to make a surprise change (e.g. add a new sound category, or change a threshold) live, to prove they understand their own code.
- The team may be asked to fix a bug that evaluators deliberately introduce.
- Hard-coding fake answers, or secretly calling an outside AI service to "cheat" the classification, is explicitly banned and can disqualify the team.
- Any AI coding assistant used (like ChatGPT or Claude) must be honestly declared in an AI_USAGE.md file, explaining what it was used for and what the students changed themselves.

9. Deliverables Checklist

This is the full list of things that must be handed in at the end, condensed from the SRS into a checklist format for project tracking.

Deliverable	Plain description
Project Report	The full written explanation of the problem, design, architecture, dataset, model results, and testing — the "story" of the whole project.
Source Code (GitHub)	All the actual code, organised in folders, with a README explaining how to run it.
Dataset	All the training/validation/testing audio clips plus a spreadsheet describing each one.
Python Model Evidence	Proof of how the Python AI model was built and how well it performs (accuracy, confusion matrix, etc.).
Google Teachable Machine Evidence	Screenshots, training details, and the exported model for the second AI.
Model Comparison Report	A spreadsheet comparing both models' answers on at least 100 unseen test recordings.
Alert Rule Files	The configuration files that define what counts as a critical alert.
Test Cases	A documented set of test scenarios proving the app handles good audio, bad audio, disagreements, etc.
Installation & Execution Instructions	A step-by-step guide so anyone else could set up and run the app from scratch.
Deployed Application	A live, publicly-reachable version of the working website.
Demonstration Video	A recorded walkthrough proving every feature actually works.
Technical Blog	A public 2,000+ word article explaining the project to outside readers.
AI Usage Declaration	Honest disclosure of any AI coding tools used during development.

10. Risks & Constraints

- **Audio quality varies wildly** — different microphones, distances, and background noise can make the same sound "look" very different to the AI. Mitigation: a diverse training dataset and the audio-quality checks built into Phase 1.
- **The two AI models may disagree often** at first, since they are trained differently. Mitigation: the manual review queue exists specifically to absorb this uncertainty safely.
- **Dataset collection is time-consuming** — 3,000 clips across ten categories is a real undertaking. Mitigation: start dataset collection on day one, in parallel with building the website.
- **Privacy & consent** — live microphone monitoring is sensitive. Mitigation: the "always show when the mic is active, never record secretly" rule is treated as a Must Have, not optional polish.
- **This is a prototype, not a certified emergency system** — the SRS is explicit that SonicSentinel AI must never present itself as an official emergency-response or law-enforcement tool. It is a competition demonstration of the underlying technology.

11. Glossary — Terms Explained for Non-Technical Readers

AI Model

A computer program that has "learned" to recognise patterns (like sound categories) by studying many labelled examples, instead of being explicitly programmed rule by rule.

Confidence Score

A percentage showing how sure the AI is about its answer. 95% confidence means the AI is very sure; 40% means it's basically guessing.

Google Teachable Machine (GTM)

A free, beginner-friendly tool from Google that lets you train a simple AI model by showing it example sounds/images, without writing complex code.

Confusion Matrix

A table that shows exactly which categories the AI mixes up with each other, e.g. how often it confused "gunshot" with "fireworks".

Precision / Recall / F1-score

Standard ways of measuring how good a model is: Precision asks "when the AI says X, how often is it really X?"; Recall asks "out of all the real X events, how many did the AI actually catch?"; F1-score blends both into one number.

Spectrogram

A colour-coded picture of sound, showing which pitches were loud at which moments in time — useful for a human to visually "see" what a sound looked like.

MoSCoW

A prioritisation technique: Must have, Should have, Could have, Won't have (for now) — used to decide what to build first.

MVP (Minimum Viable Product)

The smallest version of a product that is still genuinely useful and demonstrates the core idea, before extra polish is added.

False Positive / False Negative

A False Positive is the AI raising an alert for something that wasn't actually dangerous (a false alarm). A False Negative is the AI missing a real dangerous event (much more serious).